Date: / /

Syllabus

- Basic
- Numerical Programs
- Recursion
- Patterns
- Arrays
- Strings

Operators

1. Operation :- It is a task or an action that is perform on value based on the operator we

2. Operands :- Values on which the operands or performs.

3. Operator :- Special symbol or keywords that includes inbuild functionality based on which the task is perform on values.

Ex :- Operation : Multiplication
Operands : 9, 2
Operator : *
Expression : 9*2

4. Expression :- a statement that perform an operation

Types of Operations:

1. Arithmetic OP:
2. Relational / Comparison OP
3. Logical OP
4. Bitwise OP
5. Assignment OP
6. Membership OP
7. Identity OP

1. Arithmetic OP:

→ It is used to perform mathematical operations on number (decimal & non decimal)

-, +, *, ** [Exponentiation], // [floor div.], % [modulo]

* + :-

→ When used with a min 2 numbers it perform addition
→ When used with a single num it represent "+ve num"

Ex:- Print (10+5) # addition
Print (12.4 + 9.5 + 4 + 6) # addition
Print (+34.5) # Possitive num
Print (+8) # Possitive num.

* - :-

→ When used with a min 2 numbers it perform Subtraction
→ When used with a single num it represents "-ve num"

Ex: Print (45 - 56) # Subtraction
Print (100 - 2 - 13) # Subtraction
Print (-45.84) # negative num
Print (-7) # negative num

Snipping Tool :- Copy & Paste (Programize) (ctrl+s(Paste'))

papergrid

* *:

-> Compulsorily when used with min 2 numbers it performs "multiplication"

Eg: Print (4 * 54 * 54) # multiplication
Print (6.3 * 6 * 4.6 * 6.34) # multiplication
Print (* 7) # error
Print (* 9.3) # error

* ** [exponentiation]

-> It returns the exponent value of the provided 2 numbers.

Eg :- Print(9.5 ** 9.4)
Print (1 ** 0)

Eg :- 2^3 (3 -> Power, 2 -> Base) => 2 * 2 * 2 = 8 => Exponent value

Syn :- Base ** Power
Print (2 ** 3) # $2^3 = 8$
Print (3 ** 2) # $3^2 = 9$
Print (1.2 ** 5)
Print (5 ** 1.2)

Division op :-

1) // [floor div]

-> returns "non decimal quotient"

eg :- Print (10 // 5) # 2 (5) 10 (2, 10, 0)
Print (17 // 3) # 5 (3) 17 (5, 15, 2)
Print (22 // 7) # 3 (7) 22 (3, 21, 1)

2) float Div :-

→ It returns "Decimal quotient".

eg Print (10/5) # 2.0

```
  2
5)10
  10
  0//
```

Print (17/3) # 5.666

```
  5.66
3)17
  15
  .20.
  18
  20
  18
```

Print (11/2) # 5.5

```
  5.5
2)11
  10
  .10
  10
  0//
```

3) % Modulo

→ returns "remainder"

eg :- Print (10 % 5) # 0

```
  2
5)10
  10
  0//
```

Print (17 % 3) # 2

```
  5
3)17
  15
  .2//
```

Print (11 % 2) # 1

```
  5
2)11
  10
  1//
```

Print (2 % 11) # 2

```
  0
11)2
  0
  2//
```

Comparison / Relational Op :

→ It is used when there if a need to compare the given 'n' val's or find any kind of relativity between the val's

→ It return's Boolean val (True-1 or False-0)

→ It can be used on Boolean cond :

a. Direct val or variable holding val
b. Boolean val or anything that represents bool val
c. An expression that returns val
   →, >, <, ==, !=, >=, <=

Ex:- Print (10>5) #True
Print ((3+9)<(6+9)) #False
Print ("App" == "Pap") #False
Print ("A" != "a") #True
Print (False <= True) #True
0 1
Print (True * 0 >= (False ** 1)) #True
Print ((3 ** True) < (5+2)) #True

3) Logical OP:

→ It is used to combine and check multiple Boolean condition
→ It returns a Boolean val or anything that presents Boolean value
→ It can be used on Boolean condition:
   a) Direct value or variable holding value
   b) Boolean value or anything that represents bool value
      And, or, not

Note:- Any other value/num apart from 0 or None or "" is consider to be true.

Logical OP
Ex:-
Print (None or 0 or "") #False
Print (True or "")
Print ("" and 5) or False) #False
Print ((106 > 110) or (-1 - 3) or (7)) #7
Print (-6 or -3 or 0) #-6

a) Logical and

C_1	C_2	o/p
T	T	T
T	F	F
F	-	F

P (((65+3)<=(60+8)) and True and -3)# True
(65+3) → 68, (60+8) → 68, (68<=68) → T, True → T, -3 → T

P (10 and -5 and 6)# True
10 → T, -5 → T, 6 → T

P (16 and 0 and (6>-3))# False
16 → T, 0 → F

b) Logical or

C_1	C_2	o/p
T	-	T
F	T	T
F	F	-

P €.

c) logical not

C	o/p
T	F
F	T

Ex :- Print ((45+34) or (34<56) and -5)
Print (not (not True))
Print (-3)
Print (not (True or False or True and False))
Print (not (-4))

Bool?
→ It requires only one single operand to perform the operation.
→ It returns only Bodean value

True, False, None:
→ They are special kind of key words, bcz.
→ unlike the other key words they hold "in-built" value.

Date / /

→ These are the only 3 keywords who initially start from uppercase.

4. Bitwise OP:
1. Bitwise and (&)
2. Bitwise or (|)
3. Bitwise not / complement / negation (~) ==> requires only a single operand.
4. Bitwise XOR (^)
5. Bitwise left shift (<<)
6. Bitwise Right shift (>>)
7. Bitwise unsigned right shift (>>>)

5. Assignment OP:

→ It helps to assign (initialize), re assign (re-initialization) and copy a value present in one variable to another variable.

→ It does not return any value.

Syn:- Var_name = Value
Var_name = Initialized_Var_name
Var_name = (expression)

Component stmts

Eg 1)

```
a=25                    a → 25 → 35
b=10                    b → 10
P(a," ",b)
sum=a+b                 sum → 35
P(sum)
Print(a," ",b)
a=a+b  ⇒ a+=b
```

① Arithmetic
② Assignment

eg 2
```
i = 100
j = 2
i = i + (j * 5)  =>  i += (j * 5)
Print(i, " ", j)
```

eg 5
```
j = 3
Print(j)
j += (b + 10) => j = (b + 10)   # error
Print(j)
```

eg 3
```
a = 10
b = 2
Print(a, " ", b)
a ** b = b  =>  a = a ** b
Print(a, " ", b)
```

eg 6
```
c = 5
P(c)
c ~= 5  =>  c = (c ~= 5)   # error
Print(c)
```

eg 4
```
i = 15
Print(i)
i -= 5  =>  i = i - 5
Print(i)
```

eg 7
```
c = 5
Print(c)
c ~= c  =>  c = c ~= c   # error
Print(c)
```

eg 8
```
c = 5
Print(c)
c = ~c  =>  c = (10 + 1)   # error
Print c
```

eg 9
```
a = 3
b = 2
Print(a, "~", b)
a = a * b  =>  a *= b
Print(a, "~", b)
```

eg 10
```
a = 3
b = 2
Print(a, "~", b)
a = b - a        a = -b  =>  a = -a = b   error
Print(a, "~", b)
# It cannot be converted into Compound stmt
```

Membership

eg
```
l1 = [10, 15, 8, [2]]
P(10 in l1)
P([10] in l1)
P([] in l1)
a = 100
P(1 in a)   # error
```

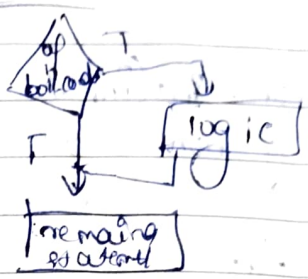
l1 → | 10 | 15 | 8 | [2] |
1001

Indenting

Flow chart

1) Conditional "if"

syn if bool_cond:
    #logic
remaining stmt.



eg:- WAL to check whether the num is +ve

```
num = int(Input("Enter a num:"))
if num > 0:
    Print(num, "is Positive")
Print("Prog exec!!!")
```

eg 1

num = 100
if num > 0: → T
    Print("100 is Positive")
Print("Prog exec!!!")

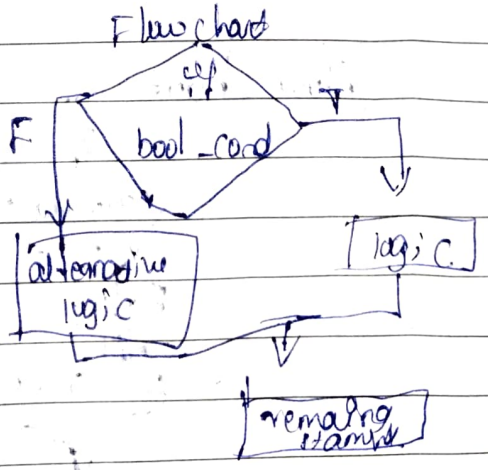
eg 2

num = -99
if num > 0: → F
Print("Prog exec!!!")

2) if..else

syn: if bool_con:
    #logic
else
    #alternate
    logic
#remaining statemt

Flow chart



Date: / /

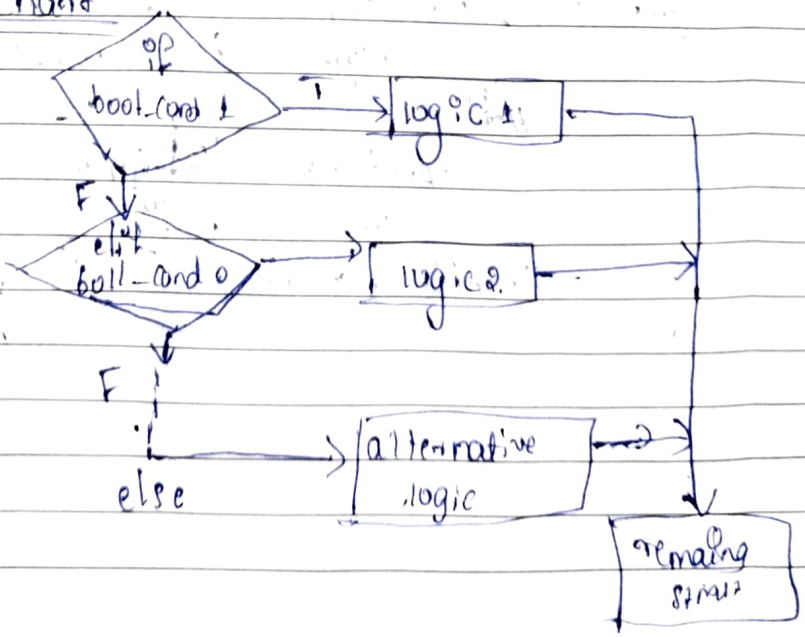
WAP to check whether it is positive or not

```
num = int(input("Enter a num:"))
if num > 0:
    print(num, "is positive")
else:
    print(num, "is not positive")
print("Prog exec!!!")
```

3) 'elif' ladder:

Syn:
```
if bool_cond1:
    # logic 1
elif bool_cond2:
    # logic 2
elif bool_cond 3:
    # logic 3
    :
else:
    # alternate logic
# remaining stmts
```

Flow chart



4) short-hand if:

Syn:
result = true_block if bool_cond else false_block

WAL to check whether the
WAL to check whether the given num is positive, negative or neutral.

Looping statements

a) "for" with range():

range():
syn :- range (start, end/stop, step)

* start => define from where the sequence should begin
  => default ~> 0

* end / stop => till where the sequence should get executed
  => it is a compulsory value.

* step => the difference between the current value and the next value.
  => default :- (+1)

	eg :- code	expected o/p	actual o/p
1)	Print(range(5))	0 1 2 3 4	range(0,5)
2)	Print(range(1,6,+2))	1 3 5	range(1,6,2)
3)	print (range (6,5)) "a"	error.	
4)	Print (range ())	error.	

papergrid

Date: / /

a) "for" with range():

Syn

for var_name in range(start, actual end, step):
    # logic
remaining stmt.

1) incre "for" loop

syn for var_name in range(start, actual end +1, positive step)
    # logic
remaining statements.

eg: Write a logic to display from 1 to n (included) (WAL)
in incrementing order. n=4

Expected o/p => 1, 2, 3, 4 (+1 between each)

start => 1 step (+1)

actual end => 4 (n)

eg: n = int(input("Enter num:"))

```
for i in range(1, n+1, +1):
    print(i, end = " ")
print("\n last updated i:", i)
```

Tracing:

n=4

1) check for the type of loop => step: +1 (positive no.) → increment loop

2) start < mentioned end   end val of loop [5]

3) for i [1] in (membership op) 1 < 5 → T → sequence val.
   print (i, e = " ")

for i [1, 2] in 2 < 5 → T

for i [4] in ✗ 5 < 5 => exit loop

papergrid

Date / /

incre ~~decre~~ . loop

Rule

1) start with positive num

2) start < mentioned n

3) The end value also need to be included then the set actual end +1

9) decre loop :-

syn for var_name in range(start, actual end -1, negative step):

#logic

remaining statement

eg. Write a logic to display n to 1 in decrementing order with a difference of 2.

Excepted value o/p :- 7, 5, 3. step [-2]

start = n

actual end = -1

eg.

```
n = int(input("enter num:"))
for i in range(n, (1-1), -2):
    print(i, end=" ")
print("\n last updated i:", i)
```

Tracking

n=7

1) step => -2 (negative step, decre. loop)

2) start > mentioned end (7>0)

3) Step 3 → end val of 'i' loop [6]

for i [7] in range(7, 0, -2) → T
    print(i, e=" ")
for i [7] in range(5, 0, -2) → T
    print(i, end=" ")

for i [1] in range(1, 0, -2) → F.
  ⇒ exit loop.
print("\n lst updated :", l)

for loop

lst = [11, -2, 3, 15]

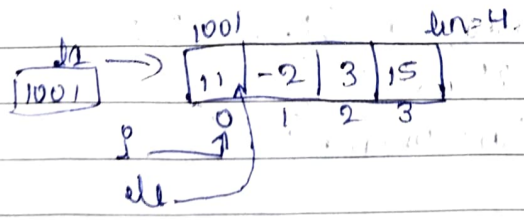
it represents index of lst.

for i in range(0, len(lst)):
    print(lst[i] * 10)

returns the ele present in the specified index of lst

Tracing:

i=0 ⇒ Print(lst[0]*10)
i=1 ⇒ " (lst[1]*10)
i=2 ⇒ " (lst[2]*10)
i=3 ⇒ " (lst[3]*10)
  ⇒ exit loop



for ele in lst:
    print(ele*10)

Tracing

ele = 11 ⇒ Print(11*10)
" = -2 ⇒ " (-2*10)
" = 3 ⇒ " (3*10)
" (15*10)

While loop:

Syn:

#initialize the var's that would be used in the loop
While bool_cond:
    #logic
    #update the looping var
#remaining stmts

Varieties of var's:

1) looping var
2) Conditional var
3) Looping + Conditional var

eg: WAP to display natural num's in reverse order

expected o/p:- n=5

① 1 2 3 4 5
② 4 5
③ 3 4 5 (n)
i ↑

```
n=int(input("Enter num:"))
i=3
while i<n:
    print(i, end="_")
    i=i+1  #i+=1
print("\n Last updated i:", i)
```

Tracing n 5
i 3 4 5 6

while i<=n: ->T
    Print(i, e="_")
    i = 3+1
while i<=n: ->T
    Print(i, e="_")
    i = 4+1
while i<=n: ->T
    Print(i, e="_")

while i<=n: ->F
=>exit loop
Print("\n Last updated i:", i)

eg: WAL to display whole number starting from 8 in descending order with a incrementing difference

n = 8

-1 -2 -3 -4 ✗

8 → 7 → 5 → 2 → -2 >0 → condition.

n (stopping var + conditional var)

```
n = int(input("Enter num:"))
diff = 1
while n>=0:
    print(n, end=" ")
    n = n - diff
    diff = diff + 1
print("\n Prog exec")
```

n 8 7 5 2 -4

diff 1 2 3 4 5.

while n>=0: → T
print(n, end=" ")
n = 8 - 1
diff = 1 + 1

while n>=0: → T
print(n, end=" ")
n = 7 - 2
diff = 2 + 1

while 5>=0: → T
print(n, end=" ")
n = 5 - 3
diff = 3 + 1

while 2>=0: → T
print(n, end=" ")
n = 2 - 4
diff = 4 + 1

while -2>=0: → F
⇒ exit loop
print("\n Progm exec")

papergrid

Date / /

1) WAC to get the I/p from the user and display it until & unless the user not entering 0. and also if the user entery 3 or 5 then ignore it

→ Excepted o/p:

Enter num : 10

The entered num is 10

```
while True:
    n = int (input ("Enter num : "))
    if n == 0:
        break
    elif n == 3 or n == 5:
        continue
    print ("The entered num is :", n)
print ("In Progm exec !!!")
```

function :

```
def func name (Parameters):   ] funcn declaratn
    # logic                   ] funcn definatn /
    return val / val's        ] body of the funcn /
                                scope of a funcn

# fun-call
var_name = func_name (arguments)
```

```
→ def substractNum (x, y):
      sub = x - y        # 7 - 3 = 4
      return sub
      print ("Hello")
  res = substractNum(7,3) # (7-3)
  Print (res * 10) #40
  Print (res * 10) #114
  res = substractNum(3,10)# (10-3) => expected O/P.
  Print (res ) # -7
```